

# 1. ПРАКТИЧЕСКАЯ РАБОТА. ОСНОВЫ РАБОТЫ С BASH-СКРИПТАМИ

**Цель практической работы:** получить практические навыки написания и использования Bash-скриптов для автоматизации задач, а также научиться разворачивать и настраивать проект с помощью скриптов, включая установку зависимостей и создание виртуального окружения.

Практическая работа состоит из 2 частей.

## 1.1. Теоретический материал для выполнения 1 части

### 1.1.1. Написание и запуск скрипта

Bash-скрипты представляют из себя последовательность команд командной строки, объединенных в один файл для решения какой-либо задачи.

Любой скрипт начинается с так называемой шебанг строки, которая указывает на расположение исполняемого файла той оболочки, для которой мы хотим написать скрипт. В нашем случае, это bash, следовательно данная строка будет выглядеть следующим образом:

```
#!/bin/bash
```

После этой строки уже следует непосредственно сам скрипт.

В рамках практической работы для написания скриптов рекомендуется создавать файлы с расширением .sh (расширение для скриптов командной оболочки в Unix-подобных операционных системах).

Напишем самый базовый скрипт, который просто будет выводить текущий каталог. Создадим файл myscript.sh и запишем в него следующий код:

```
#!/bin/bash
# This is a comment
pwd
```

Символом решетки в bash-скриптах обозначаются комментарии.

Запуск скриптов из командной строки осуществляется следующим образом — в директории с файлом скрипта необходимо написать следующую команду:

```
./myscript.sh
```

Однако просто так этого сделать не получится, система выдаст ошибку, необходимо выдать этому файлу права на выполнение:

```
chmod +x ./myscript.sh
```

После этого скрипт может быть запущен.

Для вывода текста в скриптах можно воспользоваться командой `echo`.

### ***1.1.2. Переменные***

Как и любой другой язык, Bash обладает возможностью создания переменных. Переменные бывают как пользовательскими, создаваемыми при работе скрипта, так и переменными среды, созданными для хранения параметров операционной системы.

Для обращения к переменным среды следует использовать следующую конструкцию: `$ИМЯ_ПЕРЕМЕННОЙ`. К примеру, выведем в сообщении путь к домашней директории текущего пользователя:

```
#!/bin/bash
# display user home
echo "Home for the current user is: $HOME"
```

Строковый литерал — двойные кавычки — никак не повлиял на распознавание переменной среды.

Пользовательские переменные в bash-скриптах имеют динамический тип. Вот пример их задания с последующим выводом:

```
#!/bin/bash
# testing variables
grade=5
person="Adam"
echo "$person is a good boy, he is in grade $grade"
```

В данном случае было создано 2 переменные: `grade` и `person`.

### ***1.1.3. Постановка команд***

Одна из самых полезных возможностей bash-скриптов — это возможность извлекать информацию из вывода команд и назначать ее переменным, что позволяет использовать эту информацию где угодно в файле сценария.

Сделать это можно двумя способами:

1. С помощью значка обратного апострофа «`».
2. С помощью конструкции \$().

Используя первый подход, проследите за тем, чтобы вместо обратного апострофа не ввести одиночную кавычку. Команду нужно заключить в два таких значка:

```
mydir=`pwd`
```

При втором подходе то же самое записывают другим образом:

```
mydir=$(pwd)
```

Скрипт, использующий второй подход, выглядит так:

```
#!/bin/bash
mydir=$(pwd)
echo $mydir
```

В ходе работы вывод команды `pwd` будет сохранен в переменной `mydir`, содержимое которой, с помощью команды `echo`, попадет в консоль.

#### ***1.1.4. Математические операции***

Для выполнения математических операций в файле скрипта можно использовать конструкцию вида `$(( a + b ))`:

```
#!/bin/bash
var1=$(( 5 + 5 ))
echo $var1
var2=$(( $var1 * 2 ))
echo $var2
var3=$(( $var2 / 4 ))
echo $var3
var4=$(( $var3 % 5 ))
echo $var4
var5=$(( $var4 ** 2 ))
echo $var5
```

#### ***1.1.5. Управляющая конструкция if-then-else***

Для управления потоком исполнения команд в `bash`-скриптах можно использовать управляющую конструкцию `if-then-else`. Работает она как во всех

языках программирования, однако имеет свои особенности. Вот ее подробный синтаксис:

```
if команда1
then
команды
elif команда2
then
команды
else
команды
fi
```

В отличие от большинства известных на данный момент языков `bash`-скрипты обладают возможностью задать дополнительные условия в конструкции `if-else` при помощи ключевого слова `elif`. Пример скрипта с `if-then-else`:

```
#!/bin/bash
user=anotherUser
if grep $user /etc/passwd
then
echo "The user $user Exists"
else
echo "The user $user doesn't exist"
fi
```

Пример скрипта с `elif`:

```
#!/bin/bash
user=anotherUser
if grep $user /etc/passwd
then
echo "The user $user Exists"
elif ls /home
then echo "The user doesn't exist but anyway there is a directory
under /home"
fi
```

Команда `grep` используется для поиска информации о пользователе, чье имя записано в переменной `user`, в файле `/etc/passwd`, который содержит информацию обо всех пользователях в системе.

### 1.1.6. Сравнение чисел

В отличие от привычных операций сравнения в языках программирования `bash`-скрипты обладают специфическим синтаксисом операций сравнения. Например, для чисел применяются следующие операции:

1. `n1 -eq n2` — Возвращает истинное значение, если `n1` равно `n2`.
2. `n1 -ge n2` — Возвращает истинное значение, если `n1` больше или равно `n2`.
3. `n1 -gt n2` — Возвращает истинное значение, если `n1` больше `n2`.
4. `n1 -le n2` — Возвращает истинное значение, если `n1` меньше или равно `n2`.
5. `n1 -lt n2` — Возвращает истинное значение, если `n1` меньше `n2`.
6. `n1 -ne n2` — Возвращает истинное значение, если `n1` не равно `n2`.

При использовании подобных операций выражения необходимо заключать в квадратные скобки, например:

```
#!/bin/bash
val1=6
if [$val1 -gt 5]
then
echo "The test value $val1 is greater than 5"
else
echo "The test value $val1 is not greater than 5"
fi
```

### 1.1.7. Сравнение строк

Для сравнения строк используются чуть более привычные операторы:

1. `str1 = str2` — Проверяет строки на равенство, возвращает истину, если строки идентичны.
2. `str1 != str2` — Возвращает истину, если строки не идентичны.
3. `str1 < str2` — Возвращает истину, если `str1` меньше, чем `str2`.
4. `str1 > str2` — Возвращает истину, если `str1` больше, чем `str2`.
5. `-n str1` — Возвращает истину, если длина `str1` больше нуля.
6. `-z str1` — Возвращает истину, если длина `str1` равна нулю.

Пример сравнения строк:

```
#!/bin/bash
user="likegeeks"
if [$user = $USER]
then
echo "The user $user is the current logged in user"
fi
```

При работе с операторами < и > есть несколько особенностей, которые стоит учитывать. Во-первых, их необходимо экранировать при помощи символа \. Во-вторых, необходимо заключать имена переменных в двойные кавычки "\$val2". Приведем пример работы с подобным оператором:

```
#!/bin/bash
val1=text
val2="another text"
if ["$val1" \> "$val2"]
then
echo "$val1 is greater than $val2"
else
echo "$val1 is less than $val2"
fi
```

### ***1.1.8. Проверки файлов***

Наиболее используемыми командами для bash-скриптов являются команды проверки файлов:

1. -d file — Проверяет, существует ли файл и является ли он директорией.
2. -e file — Проверяет, существует ли файл.
3. -f file — Проверяет, существует ли файл и является ли он файлом.
4. -r file — Проверяет, существует ли файл и доступен ли он для чтения.
5. -s file — Проверяет, существует ли файл и не является ли он пустым.
6. -w file — Проверяет, существует ли файл и доступен ли он для записи.
7. -x file — Проверяет, существует ли файл и является ли он исполняемым.
8. file1 -nt file2 — Проверяет, новее ли file1, чем file2.

- 9. `file1 -ot file2` — Проверяет, старше ли `file1`, чем `file2`.
- 10. `-O file` — Проверяет, существует ли файл и является ли его владельцем текущий пользователь.
- 11. `-G file` — Проверяет, существует ли файл и соответствует ли его идентификатор группы идентификатору группы текущего пользователя.

Пример скрипта, в котором используются эти команды:

```
#!/bin/bash
mydir=/home/likegeeks
if [-d $mydir]
then
echo "The $mydir directory exists"
cd $mydir
ls
else
echo "The $mydir directory does not exist"
fi
```

### ***1.1.9. Операторы цикла***

Bash-скрипты поддерживают несколько вариантов циклов для перебора последовательностей значений. В число этих циклов входят:

- а) `for`;
- б) `while`.

Для начала рассмотрим цикл `for`. Базовая структура такого цикла выглядит следующим образом:

```
for var in list
do
команды
done
```

Самый простой пример — перебор списка простых значений:

```
#!/bin/bash
for var in $first second third fourth fifth
do
echo The $var item
done
```

Для перебора сложных значений необходимо заключать эти значения в строковые литералы — двойные кавычки, к примеру:

```
#!/bin/bash
for var in first "the second" "the third" "I'll do it"
do
echo "This is: $var"
done
```

Список для цикла `for` может быть получен из результата выполнения какой-либо команды, полученного при помощи рассмотренной ранее подстановки команд. Как пример — перебор вывода команды `cat`:

```
#!/bin/bash
file="myfile"
for var in $(cat $file)
do
echo "$var"
done
```

Однако проблема такого вывода в том, что файл будет обрабатываться по словам, а не по строкам.

Это связано с тем, что оболочка `bash` считает разделителем строки следующий набор символов:

1. Пробел.
2. Знак табуляции.
3. Знак перевода строки.

Для того, чтобы задать разделитель необходимо использовать переменную окружения `IFS` (Internal Field Separator). Пример того, как можно выполнить итерацию лишь по строкам файла:

```
#!/bin/bash
file="/etc/passwd"
IFS=$'\n'
for var in $(cat $file)
do
echo "$var"
done
```

По мере работы скрипта возможно изменение переменной `IFS`, так что она может задаваться в зависимости от контекста. Для обхода файлов,



находящихся в директориях, также используется цикл `for`. Вот пример такого обхода с выводом списка файлов и директорий:

```
#!/bin/bash
for file in /home/<username>/*
do
if [-d "$file"]
then
echo "$file is a directory"
elif [-f "$file"]
then
echo "$file is a file"
fi
done
```

Как можно видеть из примера для обхода файлов можно использовать следующий путь, который подразумевает сбор всех файлов и директорий — `/home/<username>/*`.

Вторая разновидность циклов — цикл `while`:

```
while команда проверки условия
do
другие команды
done
```

Пример скрипта с таким циклом:

```
#!/bin/bash
var1=5
while [$var1 -gt 0]
do
echo $var1 var1=${var1 - 1}
done
```

Для управления циклами также могут применяться привычные команды операторы `break` и `continue`. Результаты работы цикла также могут быть выведены в файл при помощи перенаправления вывода. Вот пример такого скрипта:

```
#!/bin/bash
for ((a = 1; a < 10; a++))
do
echo "Number is $a"
done > myfile.txt
echo "finished."
```

## 1.2. Теоретический материал для выполнения 2 части

Bash-скрипты позволяют создать с нуля все необходимое окружение в системе, начиная с установки самого python и всего необходимого ПО для запуска приложения и заканчивая запуском самого приложения.

В результате выполнения 2 части практической работы должен быть написан и запущен единый bash-скрипт, который автоматизирует процесс настройки и запуска Django-проекта.

Проект находится по следующей ссылке:

<https://www.dropbox.com/scl/fi/xy8atv1u2408r8jao5xov/blocknote-master.tar?rlkey=tmlba0d4cu2eifq3shsknsc0y&st=w74ucvx1&dl=0>.

Перед началом работы необходимо убедиться, что в системе установлены все необходимые пакеты:

1. Проверить наличие Python 3 (команда `python3 --version`).
2. Если Python не установлен, добавить его установку через пакетный менеджер (`apt` для Ubuntu/Debian, `yum` для CentOS).
3. Убедиться, что установлены `pip` и `venv` (модуль для создания виртуального окружения).
4. Проверить наличие `curl` или `wget` для загрузки проекта.

### 1.2.1. Скачивание и распаковка архива

Как было уже описано выше, для загрузки проекта необходимо использовать одну из команд: `curl` или `wget`. Как пример, применение команды `curl` выглядит следующим образом:

```
curl -L -o project.tar "URL"
```

В данной команде флаг `-L` указывает `curl` следовать за перенаправлениями и получать содержимое из конечного пункта, флаг `-o project.tar` сохраняет скачанный файл на компьютер под указанным именем.

После скачивания файла необходимо его распаковать. Это возможно сделать с использованием команды:

```
tar -xvf project.tar -C /path/to/directory
```

Флаг -C /path/to/directory распаковывает архив в указанную папку (если не указать, то распакует в текущую). Также в этой команде используются следующие флаги:

- а) -x (extract) — распаковать архив;
- б) -v (verbose) — показывает список распаковываемых файлов (для удобного отслеживания процесса);
- в) -f (file) — указать имя файла архива (project.tar).

### ***1.2.2. Определение зависимостей проекта***

Любой проект зависит от ряда библиотек, которые предоставляют тот или иной функционал. Для развертывания приложения необходимо, чтобы данные библиотеки были установлены в том окружении, где предполагается это самое развертывание.

Проект написан на языке программирования Python. Необходимо составить список зависимостей проекта в виде requirements.txt файла. Данный файл содержит в себе список библиотек, которые необходимо установить в окружение для запуска приложения.

Существует несколько вариантов поиска зависимостей проекта. Можно выбрать любой из них и подробно расписать шаги выполнения по одному из них.

*Примечание: любой выбранный метод должен отразиться в итоговом скрипте запуска кроме ручного создания.*

1. Ручное создание requirements.txt (для ознакомления).

Для поиска зависимостей необходимо самостоятельно создать файл requirements.txt в корне проекта, далее вручную перечислить в файле все зависимости с фиксированными версиями. Каждый пакет необходимо указывать на новой строке в формате «имя==версия» (без кавычек).

Зависимости в Python можно определить по import'ам в файлах, однако некоторые библиотеки включены в стандартную библиотеку языка, поэтому также необходимо будет определить, является ли библиотека внешней или же встроенной в язык.

2. Автоматический сбор через pip freeze.

Команда `pip freeze` фиксирует все установленные пакеты в системе/окружении.

Обычно команда используется следующим образом:

```
pip freeze > requirements.txt
```

Данное использование команды создает файл `requirements.txt`, в котором перечисляет все установленные пакеты Python и их версии в текущей среде.

Ограничение команды `pip freeze > requirements.txt` заключается в том, что она экспортирует полный список установленных пакетов с версиями из текущей виртуальной среды, поэтому лишние библиотеки могут повлиять на корректную работу проекта.

Также, чтобы избежать возможных ошибок, перед использованием `pip freeze` рекомендуется активировать виртуальную среду (пункт 2.3.3).

3. Анализ импортов в коде через `grep` (полуавтоматический сбор зависимостей).

Метод `grep` можно настроить для поиска строк ключевыми словами `import` и `from` во всех `.py` файлах. Как пример, поиск импортов можно произвести через следующую команду:

```
grep -rh --include='*.py' -E '^\\s*(import|from)\\s+' .
```

В данной команде используются следующие флаги:

- а) `-r` (рекурсивный поиск) — просматривает все файлы в указанной директории и поддиректориях;
- б) `-h` (скрытие имен файлов) — не показывает имена файлов в результатах. Без этого флага вывод был бы в формате: `src/main.py:import os`;
- в) `--include='*.py'` — ограничивает поиск только по файлам с расширением `.py`;
- г) `-E '\\s*(import|from)\\s+'` (расширенные регулярные выражения для поиска слов `import` и `from`).

Использование этой команды предполагает также добавление команд для извлечения имен пакетов через `awk`, сортировки и удаления дубликатов через `sort -u`. И также необходимо полученные библиотеки записать в файл через `> requirements.txt`.

При использовании данного метода `grep` может возникнуть ситуация неполного написания имен пакетов, а также добавления встроенных модулей Python (которые не нужны для проекта). Для этого в итоговом скрипте запуска

необходимо использовать команду `sed -i`, которая позволит отредактировать файл `requirements.txt` в консоли без изменения исходного файла, но при этом обработать его, удалить лишние модули и исправить имена пакетов на корректные.

#### 4. Автоматизированный поиск зависимостей через `pipreqs`.

Для использования данного метода необходимо установить `pipreqs` (если не установлен ранее) через `pip install pipreqs`. После этого необходимо перейти в директорию проекта, где находится код на Python и выполнить команду:

```
pipreqs .
```

Инструмент просканирует директорию проекта, проанализирует операторы импорта в коде и создаст файл `requirements.txt`.

Некоторые сложности, связанные с использованием `pipreqs`:

1. Не всегда точный результат, так как инструмент анализирует только импорты в исходном коде. Если какие-либо зависимости устанавливаются динамически или используются только в процессе выполнения кода, они могут быть пропущены. В таких случаях рекомендуется вручную проверить и дополнить получившийся файл `requirements.txt`.

2. Возможны проблемы с кодировкой `utf-8` и `utf-8-sig`.

3. По умолчанию `pipreqs` игнорирует папку с виртуальным окружением `.venv` (или `.env`). Чтобы создать файл `requirements.txt` только с импортированными пакетами, можно использовать команду `pipreqs --ignore .venv`.

### ***1.2.3. Создание виртуального окружения***

Python позволяет создавать так называемое виртуальное окружение. Данное окружение представляет из себя отдельную копию Python с собственным набором библиотек. Оно позволяет работать с проектами в изолированном окружении, чтобы избежать конфликтов версий.

Следующая команда создаст папку с указанным именем (в данном случае «`venv`») и настроит виртуальное окружение Python в этой папке:

```
python -m venv .venv // Windows  
python3 -m venv .venv // Linux и macOS
```

Виртуальное окружение изолирует установленные пакеты и зависимости для проекта, позволяя им существовать в отдельном пространстве и предотвращая конфликты между версиями пакетов в разных проектах.

После создания виртуального окружения можно активировать его, используя команду в зависимости от операционной системы:

- В Windows: `venv\Scripts\activate.bat`;
- В macOS и Linux: `source venv/bin/activate`.

*Примечание: рекомендуется проверить синтаксис команды активации виртуального окружения для Вашей операционной системы, поскольку есть разница в ее использовании в Windows (cmd), в Windows (PowerShell) и в Linux/macOS.*

#### **1.2.4. Установка зависимостей**

Обязательная установка найденных зависимостей из файла `requirements.txt` возможно с использованием команды:

```
pip install -r requirements.txt
```

#### **1.2.5. Написание итогового скрипта**

Основные требования по содержанию итогового скрипта следующие:

1. Автоматическая установка Python и `pip`, если они не установлены (а также, при необходимости, их обновление).
2. Скачивание и распаковка проекта.
3. Поиск зависимостей одним из вышеперечисленных способов.

*Примечание: при написании итогового скрипта используется 2, 3 или 4 методы.*

4. Создание и активация виртуального окружения.
5. Установка зависимостей.
6. Запуск миграций и сервера Django.

Запуск миграций и сервера возможен с использованием следующих команд:

```
python manage.py makemigrations  
python manage.py migrate  
python manage.py runserver
```

Итоговый скрипт должен запустить приложение Блокнота по адресу (оно отобразится, если все действия были сделаны корректно). Необходимо

перейти в приложение, зарегистрироваться и написать в открывшейся заметке свое ФИО и группу.

### **1.3. Практическая часть**

1 часть. Написание Bash-скриптов для автоматизации задач.

Для выполнения 1 части необходимо написать bash-скрипты к следующим задачам:

1. Напишите сценарий, который выводит дату, время, список зарегистрировавшихся пользователей и uptime системы и сохраняет эту информацию в файл.
2. Напишите сценарий, который выводит содержимое любого каталога или сообщение о том, что его не существует.
3. Напишите сценарий, который с помощью цикла прочитает файл и выведет его содержимое.
4. Напишите сценарий, который с помощью цикла выведет список файлов и директорий из текущего каталога, укажет, что есть файл, а что — директория.
5. Напишите сценарий, который подсчитает объем диска, занимаемого директорией. В качестве директории можно выбрать любую директорию в системе.
6. Напишите сценарий, который выведет список всех исполняемых файлов в директории, для которых у текущего пользователя есть права на исполнение.

2 часть. Развертка и запуск проекта при помощи Bash Script.

Для выполнения 2 части необходимо выполнить задания, опираясь на пункты из разделов 1.2.1–1.2.5:

1. Определение зависимостей проекта.
2. Создание виртуального окружения.
3. Написание скрипта запуска приложения.

### **1.4. Вывод**

Практическая работа направлена на освоение ключевых навыков написания Bash-скриптов и автоматизации задач в Linux-среде. Это позволит создавать скрипты для мониторинга, обработки файлов и настройки окружения, что значительно ускоряет разработку и развертывание проектов.

### **1.5. Контрольные вопросы**

1. Что представляет из себя Bash-скрипт?

2. Что такое shebang (`#!/bin/bash`) и зачем он нужен в Bash-скриптах?
3. Как сделать скрипт исполняемым в Linux? Приведите команду.
4. Как объявить переменную в Bash? Приведите пример с выводом её значения.
5. Что делает команда `pwd` в Bash?
6. Как сохранить вывод команды `pwd` в переменную? Приведите два способа.
7. Как работает конструкция `if-then-else` в Bash?
8. Какие операторы сравнения используются для чисел (`-eq`, `-gt`)?
9. Какая команда проверяет, является ли объект директорией?
10. Что означает IFS (Internal Field Separator)?
11. Какая команда проверяет, существует ли файл и доступен ли он для записи?
12. Какая команда проверяет, существует ли файл и является ли он исполняемым?
13. Что такое виртуальное окружение?
14. Как создать виртуальное окружение в Python?
15. Для чего нужен файл `requirements.txt`? Как его создать?

### 1.6. Требования к отчету

По итогу выполнения практической работы необходимо оформить отчет, включающий:

1. Титульный лист.
  2. Оглавление.
  3. Цели и задачи работы.
  4. Описание выбранного метода поиска зависимостей проекта (можно использовать 2, 3 и 4 метода) со скриншотами. Обязательно приложить скриншот полученного файла `requirements.txt`.
  5. Написание итогового скрипта.
  6. Запуск приложения по адресу, регистрация и добавление своего ФИО и группы в заметку.
  7. Письменные развернутые ответы на все контрольные вопросы.
  8. Вывод о проделанной работе.
- Отчет о практической работы необходимо загрузить в Систему дистанционного обучения и очно защитить на паре.



### 1.7. Список источников для изучения

1. Как создать и установить файл requirements.txt в Python [Электронный ресурс] // Merionet. – Режим доступа: <https://wiki.merionet.ru/articles/requirementstxt-kak-sozdat-i-ustanovit-fail-requirementstxt-v-python> (дата обращения: 21.04.2025).
2. Использование pipreqs и pip-tools для генерации файла requirements.txt [Электронный ресурс] // Dev-Gang. – Режим доступа: <https://dev-gang.ru/article/prosteishii-sposob-izvleczenija-faila-requirementstxt-na-python-752ryn0mfi/> (дата обращения: 21.04.2025).
3. Виртуальное окружение Python (venv) [Электронный ресурс] // Pythonchik. – Режим доступа: <https://pythonchik.ru/okruzhenie-i-pakety/virtualnoe-okruzhenie-python-venv> (дата обращения: 21.04.2025).

## 2. ПРАКТИЧЕСКАЯ РАБОТА. СИСТЕМА КОНФИГУРАЦИОННОГО УПРАВЛЕНИЯ ANSIBLE

**Цель практической работы:** получить навыки настройки вычислительной инфраструктуры при помощи системы конфигурационного управления Ansible.

### 2.1. Описание выполнения работы

#### 2.1.1. Подготовка инфраструктуры

Прежде всего, необходимо создать 2 виртуальные машины Ubuntu или Debian. Этими машинами необходимо будет управлять в ходе практики. Имя пользователя должно быть указано как `user`. Это важно для дальнейшей работы. В качестве образа операционной системы (ОС) рекомендуется использовать дистрибутив Debian, который можно загрузить по ссылке: <https://www.debian.org/distrib/>.

Также при установке пакетов необходимо обязательно выбрать SSH-сервер.

*Примечание: если Ваш компьютер недостаточно мощный, можно развернуть одну виртуальную машину.*

После создания и установки ОС необходимо настроить доступ к сети. Для начала в VirtualBox измените тип сетевого подключения на сетевой мост. Это позволит Вашей виртуальной машине находиться в локальной сети наравне с физическими устройствами (рис. 5.1). Имя сетевого моста у каждого будет отличаться.

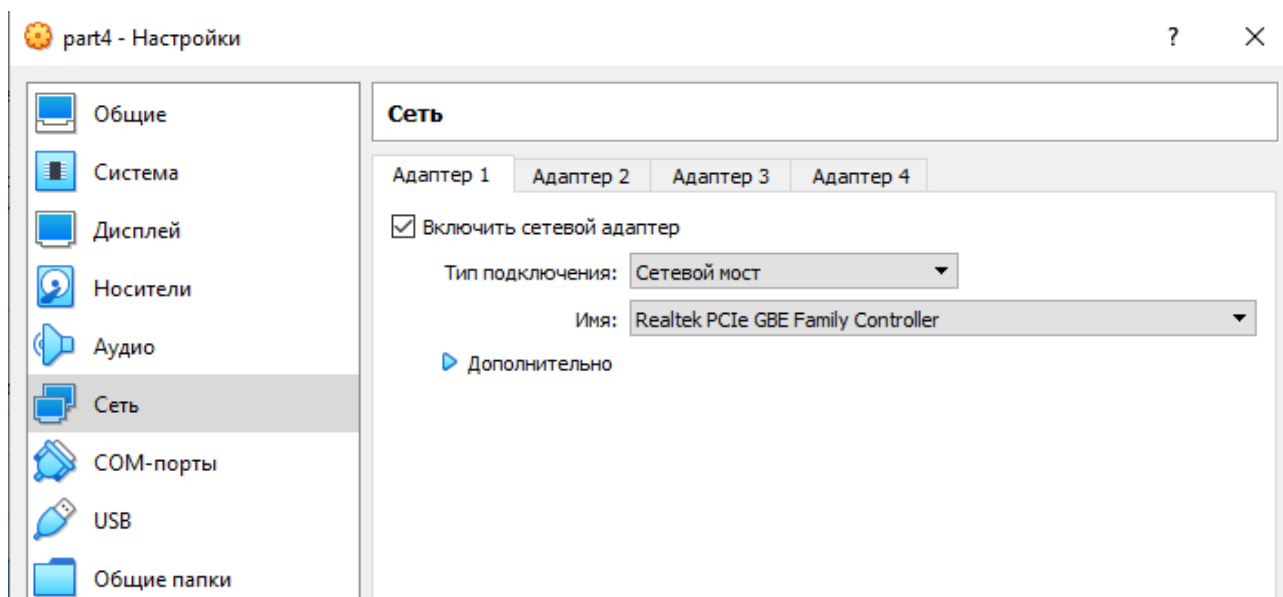


Рисунок 5.1. Изменение параметров сети

Теперь необходимо настроить ip-адрес для каждой машины.

Сначала необходимо узнать адрес той сети, в которой расположена хостовая машина. Сделать это можно, зайдя в консоль Windows и прописав команду:

```
ipconfig
```

Для пользователей Linux есть команда:

```
hostname -I
```

Для пользователей MacOS команды:

```
ipconfig getifaddr en0 //при подключении по Wi-fi  
ipconfig getifaddr en1 //при подключении по Ethernet
```

Теперь можно приступить к настройке виртуальных машин.

*Примечание: настройку необходимо выполнять под пользователем root.*

Будем производить настройку при помощи файла /etc/network/interfaces. Открыв его в редакторе nano, можно увидеть набор параметров для двух интерфейсов lo и что-то похожее на enp0s3. Второй интерфейс необходимо настроить по следующему подобию:

```
allow-hotplug enp0s3  
iface enp0s3 inet static  
address 192.168.1.5  
mask 255.255.255.0  
gateway 192.168.1.1
```

Параметр address назначается по ip, полученному для сети хостовой машины. К примеру, по ip 192.168.1.220 физической машины, можно настроить ip 192.168.1.5 для виртуальной машины.

Параметр mask обычно равен 255.255.255.0.

Параметр gateway чаще всего является первым адресом в сети.

После заполнения файла перезапустите машину командой reboot.

Еще одним важным шагом является установка ssh-сервера, который позволит удаленно подключаться к машине. Установить его можно при помощи команды:

```
apt install ssh
```

Также необходимо создать управляющую машину, на которой непосредственно будет установлен Ansible. Это может быть как виртуальная машина, тогда необходимо будет проделать те же самые манипуляции, что и для управляемых машин, так и хостовая машина. Для пользователей Windows рекомендуется воспользоваться WSL2, который позволяет поднять Linux прямо на хостовой машине. Установку можно воспроизвести по ссылке: <https://learn.microsoft.com/ru-ru/windows/wsl/install>.

После этого можно переходить к конфигурации машин под Ansible.

### 2.1.2. Установка Ansible

Ansible — это инструмент автоматизации без агентов, который Вы устанавливаете на одном хосте (называемом узлом управления).

С управляющего узла Ansible Вы можете удаленно управлять целым парком машин и других устройств (называемых управляемыми узлами) с помощью SSH, удаленного управления Powershell и множества других транспортных средств, и все это с помощью простого интерфейса командной строки, не требующего баз данных или демонов. Руководство по эксплуатации можно найти по ссылке: [https://docs.ansible.com/ansible/latest/installation\\_guide/intro\\_installation.html#id22](https://docs.ansible.com/ansible/latest/installation_guide/intro_installation.html#id22).

Команда для установки Ansible в дистрибутивах Linux на базе Debian/Ubuntu выглядит следующим образом:

```
sudo apt install ansible
```

Если Вы являетесь уже root-пользователем, то sudo не нужна.

*Примечание: настройку необходимо выполнять под пользователем root.*

Также можно установить Ansible при помощи пакетного менеджера Python 3 (поскольку Ansible написан на Python). Более подробно можно ознакомиться по ссылке: [https://docs.ansible.com/ansible/latest/installation\\_guide/intro\\_installation.html#id22](https://docs.ansible.com/ansible/latest/installation_guide/intro_installation.html#id22).

Наличие Ansible можно проверить командой:

```
ansible --version
```

После чего необходимо подключиться к управляемым машинам при помощи ssh (ip\_address — адрес управляемого узла в сети):

```
ssh root@ip_address
```

Если соединение удастся установить, после подключения система попросит ввести пароль пользователя.

Если подключения не будет, необходимо разрешить доступ на машину от пользователя root. Для этого на управляемых машинах необходимо открыть файл /etc/ssh/sshd\_config, найти строку PermitRootLogin, раскомментировать ее и изменить значение на значение «yes». Итоговая строка должна выглядеть так (рис 5.2):

A screenshot of a terminal window showing the /etc/ssh/sshd\_config file being edited with nano 7.2. The file content is as follows:

```
GNU nano 7.2 sshd_config
# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none
```

*Рисунок 5.2. Файл /etc/ssh/sshd\_config*

После этого необходимо перезапустить службу ssh при помощи команды:

```
service ssh restart
```

Теперь на машину должен быть доступ.

Чтобы выйти из ssh-соединения необходимо прописать команду exit.

Далее необходимо сгенерировать ключ при помощи команды:

```
ssh-keygen
```

Эта команда создаст ключ для возможности подключения к удаленному хосту без пароля. После этого необходимо воспользоваться командой `ssh-copy-id` для передачи ключа на обе виртуальные машины:

```
ssh-copy-id root@ip_address
```

Также на каждом управляемом хосте необходимо установить утилиту `sudo` (если не была установлена ранее). Для этого необходимо сначала залогиниться под суперпользователем при помощи команды:

```
su -
```

Либо же при старте системы сразу выбрать пользователя `root`. После чего необходимо ввести команду:

```
apt install sudo
```

После выполнения всех перечисленных манипуляций инфраструктура готова к управлению через Ansible.

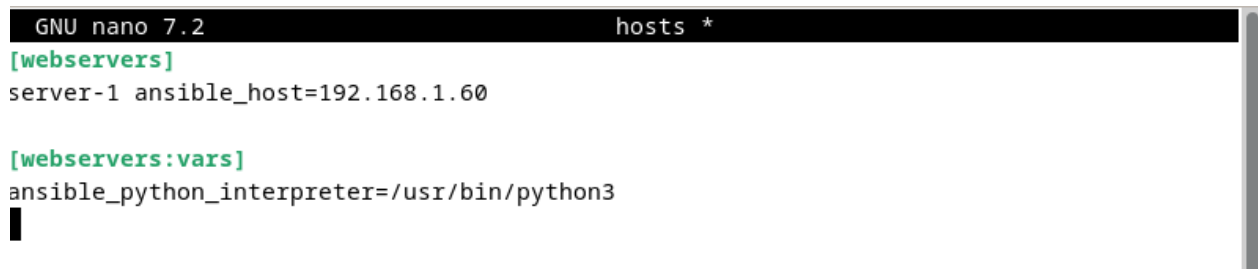
Вся настройка самого Ansible может производиться через файл `/etc/ansible/ansible.cfg`

### ***2.1.3. Настройка Ansible***

Все действия также необходимо выполнять под пользователем `root`.

Настроим `inventory`-файл. Создадим директорию `ansible` в домашнем каталоге пользователя для хранения репозитория инфраструктуры, будем считать эту директорию рабочей и в ней создадим файл `hosts`.

Заполните файл по образцу рис. 5.3. IP-адрес приведен только для примера. Если Вы настраиваете несколько управляемых машин, то их нужно называть `server-2`, `server-3` и так далее.



```
GNU nano 7.2          hosts *
[webservers]
server-1 ansible_host=192.168.1.60

[webservers:vars]
ansible_python_interpreter=/usr/bin/python3
```

*Рисунок 5.3. Настройка файла hosts*

В файле hosts:

1. `webservers` — это название группы хостов, управляемых при помощи Ansible.
2. `webservers:var` — список переменных для группы хостов `webservers`.

В указанном примере задается переменная `ansible_python_interpreter`, которая указывает, какой интерпретатор `python` должен использоваться на управляемых узлах. Без указания этой переменной может возникнуть ошибка, если хост в качестве интерпретатора по умолчанию использует `python` второй версии, а не третьей.

Inventory-файл содержит в себе всю необходимую информацию о хостах, которые должны управляться системой Ansible.

Выполним самую первую команду для проверки работы Ansible:

```
ansible -i ./hosts -m ping all
```

Данная команда выполнит команду `ping` для всех хостов в `inventory` и выдаст результат выполнения. Конкретную команду на хостах можно выполнить при помощи следующей конструкции:

```
ansible -i ./hosts -m command -a free all
```

Данная команда покажет объемы свободной оперативной памяти на всех хостах. Вместо `all` можно указать имя хоста из файла `hosts`, чтобы выполнить команду на конкретной машине. Попробуйте самостоятельно выполнить команду `uptime` на одной из машин.

#### **2.1.4. Факты**

Факты — это параметры, которыми можно управлять в реализуемых сценариях.

Соберем возможные факты с управляемого хоста `server-1`:

```
ansible server-1 -i ./hosts -m command -m setup
```

### 2.1.5. *Playbook*

Playbook — это конфигурационный сценарий, написанный на языке YAML, который впоследствии будет выполняться на управляемых хостах.

Необходимо создать файл с именем `playbook` (файлы `playbook` в Ansible не имеют строгого обязательного расширения, но рекомендовано использовать `.yaml`), который установит веб-сервер Nginx на управляемые хосты. Следующий текст необходимо внести в файл:

```
---
- name: Install Nginx to Webservers
  hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx
      apt:
        name=nginx
        state=present
        update_cache=yes
      when:
        ansible_os_family == "Debian"
      notify:
        - Nginx Systemd
  handlers:
    - name: Nginx Systemd
      systemd:
        name: nginx
        enabled: yes
        state: started
```

Сам файл с `playbook` состоит из сценариев. Каждый сценарий начинается с ключевого слова `name`.

Раздел `tasks` — это список задач, которые необходимо выполнить на управляемой машине в ходе сценария. Каждый элемент списка задач начинается с символа дефис (-). Этот символ в языке YAML обозначает начало ассоциативного массива.

При помощи ключевого слова `name` задается имя задачи. При помощи ключевого слова `when` задаются условия для запуска задачи. К примеру, в



указанном playbook задается условие, что задача по установке nginx должна выполняться только на хосте под управлением Debian.

Раздел `handlers` — это действия, которые будут выполняться после завершения задачи. Они запускаются на выполнение только 1 раз и после завершения всего сценария.

Ключевое слово `notify` позволяет запускать `handlers`. В указанном примере после установки nginx будет запущен `handler`, активирующий автозапуск сервиса nginx.

После написания playbook его можно выполнить при помощи следующей команды:

```
ansible-playbook -i hosts <имя файла playbook>
```

В результате успешного запуска Вы должны увидеть следующий вывод в терминале (рис. 5.4):

```
PLAY [Install Nginx to Webservers] *****

TASK [Gathering Facts] *****
Enter passphrase for key '/root/.ssh/id_rsa':
ok: [server-1]

TASK [Install Nginx] *****
changed: [server-1]

RUNNING HANDLER [Nginx Systemd] *****
ok: [server-1]

PLAY RECAP *****
server-1 : ok=3    changed=1    unreachable=0    failed=0    skipped=0
0    rescued=0    ignored=0
```

*Рисунок 5.4. Результат выполнения playbook*

Если же попробовать запустить выполнения playbook еще раз, никаких изменений произведено не будет, а будут выведены лишь статусы `ok`, означающие соответствие состояния систем с указанным playbook.

После этого можно перейти на любой из двух узлов (при помощи `ssh` или же непосредственно открыв окно виртуальной машины) и убедиться в том, что nginx действительно установлен при помощи команды:

```
systemctl status nginx
```

Теперь остановим и удалим с данного узла nginx при помощи следующих команд непосредственно через терминал:

```
systemctl stop nginx  
apt remove nginx
```

Теперь повторно необходимо запустить playbook. В результате выполнения нужно увидеть, что на первом сервере вновь запущен nginx.

#### ***2.1.6. Более сложный Playbook***

Необходимо создать второй файл для написания сложного playbook. Его содержание должно быть следующим:

```
---
- name: Install and config Nginx
  hosts: webservers
  become: yes

  vars:
    html_dir: /usr/share/nginx/html
    greeting: "Hello Everybody!"
  tasks:
    - name: Install Nginx
      apt:
        name=nginx
        state=present
        update_cache=yes
      when:
        ansible_os_family == "Debian"
      notify:
        - Nginx Systemd

    - name: Delete default HTML files
      shell: /bin/rm -rf /usr/share/nginx/html/*.html

    - name: Replace config file
      vars:
        nginx_user: user
        worker_processes: 2
        worker_connections: 256
      template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        mode: 0644
      register: result
      failed_when: result.failed == true
      notify: Reload Nginx
```

```

- name: Copy index file
  copy: src=files/index.html dest={{ html_dir }} mode=0644
  notify: Reload Nginx

- name: Generate dynamic HTML from template
  template:
    src=templates/hello.html.j2 dest={{ html_dir }}/hello.html
  owner=root mode=0644
  notify: Reload Nginx

handlers:
- name: Nginx Systemd
  systemd:
    name: nginx
    enabled: yes
    state: started
- name: Reload Nginx
  systemd: name=nginx state=reloaded

```

В разделе `vars` описываются переменные, которые затем можно использовать при запуске задач. Для использования переменных в задачах используется синтаксис шаблонизатора — `{{ имя_переменной }}`.

Еще одним инструментом является использование шаблонных файлов. Для этого используется шаблонизатор Jinja2. Шаблонные файлы добавляются в директорию `templates` с дополнительным расширением `j2`, то есть итоговое имя файла должно выглядеть, например, следующим образом `index.html.j2`.

При реализации вышеописанного `playbook` используется 2 шаблонных файла: `nginx.conf.j2`, `hello.html.j2` (задачи «Replace config file» и «Generate dynamic HTML from template»).

Использовать шаблонизированные файлы можно при помощи модуля `template`. В качестве параметров задается `src` — путь к шаблонному файлу, `dest` — путь к файлу на управляемом узле, `mode` — режим доступа в 8-ричной системе счисления.

Помимо этого, используется модуль `register`, позволяющий получить и проверить результат выполнения команды. Все результаты записываются в переменную `result` и при истинном значении параметра `failed`, означающего, что процесс переноса шаблона провалился, весь `task` становится проваленным благодаря конструкции `failed_when`.

Для копирования статического файла используется модуль `copy`. Ему так же задается параметр `src`, указывающий путь до копируемого файла, параметр

dest, указывающий путь для файла на целевом хосте, и параметр mode, указывающий режим доступа. Следует отметить, что параметры могут быть записаны в одну строку, как это сделано в случае сложного playbook.

В модуле template также могут быть заданы параметры owner и group, обозначающие владельца и группу для переносимого файла.

Для запуска сложного playbook необходимо создать папку templates (должна лежать с playbook в одной директории), а затем добавить 2 шаблонизированных файла nginx.conf.j2 и hello.html.j2.

Содержание hello.html.j2 следующее:

```
"Server {{ ansible_hostname }}
( ip {{ansible_default_ipv4.address }}
) greets you: {{ greeting | default("Hello") }}"
```

Содержание nginx.conf.j2 следующее:

```
# Пользователь, из-под которого будет запущен nginx
user {{ nginx_user }};

# Количество рабочих процессов, которые будет задействовать nginx
worker_processes {{ worker_processes }};
error_log /var/log/nginx/error.log warn;
pid /var/run/nginx.pid;
events {
# Число соединений, которое может поддерживать каждый процесс
    worker_connections {{ worker_connections }};
}
http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    keepalive_timeout 60;
    gzip on;

    server {
        access_log off;
        location / {
            root /usr/share/nginx/html;
            try_files $uri $uri/ $uri.html /index.html;
        }
    }
}
```

Также необходимо создать папку `files`, в которой будут храниться все статические файлы, к примеру файл `index.html`, которым будут заменены стандартные файлы `nginx`.

Содержимое файла `index.html`:

```
Web server is working!
```

В результате содержимое директории `ansible` должно быть следующим (рис 5.5):

```
├── files
│   └── index.html
├── hosts
├── playbook
├── playbook2
└── templates
    ├── hello.html.j2
    └── nginx.conf.j2
```

3 directories, 6 files

Рисунок 5.5. Содержимое директории `ansible`

Теперь выполним команду для запуска `playbook` с пробным прогоном, который позволит проверить корректность написанного `playbook` без внесения изменений на целевые узлы:

```
ansible-playbook -i hosts <имя файла playbook> --check
```

*Примечание: при использовании флага `--check` Ansible задействует модуль `python`, которого может не оказаться на управляемых узлах. Для его установки необходимо выполнить следующую команду:*

```
ansible -i hosts -m "apt name=python-apt state=latest" all
```

Вывод покажет, какие изменения будут произведены на узле после применения данной команды, однако, на текущий момент, никаких изменений произведено не было. В этом можно убедиться, обратившись к одному из узлов

при помощи утилиты curl для получения базовой html страницы nginx. Сделать это можно командой (IP-адрес приведен в качестве примера):

```
curl -L http://192.168.1.6
```

Как можно видеть, полученная страница отличается от той, что должна была быть загружена при помощи Ansible.

Помимо этого, можно задать выполнение playbook только на одном узле при помощи флага --limit. Например, следующая команда позволит выполнить проверку только на втором сервере:

```
ansible-playbook -i hosts <имя файла playbook> --check --limit  
server-2
```

Теперь внесем изменения в конфигурацию первого сервера, убрав ключ -check из предыдущей команды, а затем запросим базовую страницу при помощи все той же утилиты curl: curl -L <http://192.168.1.6>.

Также можно запросить созданную при помощи шаблона страницу hello:

```
curl -L http://192.168.1.6/hello
```

Как можно увидеть, изменения успешно применились. Можете самостоятельно поменять переменную greetings, затем загрузить повторно playbook на машину и понаблюдать за изменениями.

После этого применим playbook для всех серверов в группе.

Как можно заметить, при небольшом расширении задачи, решаемой при помощи Ansible, файл playbook начинает разрастаться, что вызывает неудобство при его чтении.

### ***2.1.7. Ролу Ansible***

Данный механизм позволяет систематизировать конфигурации путем выделения каждого механизма (задач, переменных, обработчиков и т.д.) в отдельные части.

Ansible имеет похожий на GitHub сервис, называемый Ansible Galaxy. Там находится множество ролей для Ansible, которыми можно воспользоваться. Для использования данного сервиса в Ansible встроена команда ansible-galaxy.

Загрузим в систему роль для установки MySQL при помощи команды:

```
ansible-galaxy install geerlingguy.mysql
```

Найти файлы установленной роли можно по следующему пути `~/.ansible/roles/geerlingguy.mysql/`. Рассмотрим более подробно содержимое данной роли. При помощи следующей команды можно вывести дерево каталога для большей наглядности:

```
tree .
```

Если команда `tree` не поддерживается, ее можно установить при помощи команды `apt install tree`.

Вывод команды должен отобразить следующее содержимое (рис 5.6):

```
root@anna2:/home/anna2/ansible# cd /
root@anna2:/# cd ~/.ansible/roles/geerlingguy.mysql/
root@anna2:~/.ansible/roles/geerlingguy.mysql# tree
.
├── defaults
│   └── main.yml
├── handlers
│   └── main.yml
├── LICENSE
├── meta
│   └── main.yml
├── molecule
│   └── default
│       ├── converge.yml
│       └── molecule.yml
├── README.md
└── tasks
    ├── configure.yml
    ├── databases.yml
    ├── main.yml
    ├── replication.yml
    ├── secure-installation.yml
    └── setup-Archlinux.vml
```

*Рисунок 5.6. Содержимое роли `geerlingguy.mysql`*

Назначения директорий в роли следующие:

1. Директория `defaults` содержит значения переменных по умолчанию.
2. Директория `handlers` содержит описание обработчиков.



3. Директория `meta` содержит информацию о роли: создатель роли, ее описание, используемая лицензия, зависимости и т.д. Данный файл используется в дальнейшем для Ansible Galaxy.

4. Директория `molecule` содержит сценарии тестирования для роли Ansible.

5. Директория `tasks` содержит непосредственно сценарии для конфигураций.

6. Директория `templates`, как можно догадаться, содержит шаблонные параметризованные файлы.

7. Директория `vars` содержит файлы описания различных переменных.

Попробуем создать свою роль для установки ранее разобранных playbook `nginx`. Сперва вернемся в рабочую папку со всеми файлами для Ansible и создадим там директорию `roles`. После чего перейдем в эту директорию и иницируем роль стандартной структуры при помощи команды:

```
ansible-galaxy init nginx
```

При помощи команды `tree` опять же можно посмотреть структуру директории (рис. 5.7):

```
└─ nginx
   ├── defaults
   │   └─ main.yml
   ├── files
   │   └─ index.html
   ├── handlers
   │   └─ main.yml
   ├── meta
   │   └─ main.yml
   ├── README.md
   ├── tasks
   │   └─ main.yml
   ├── templates
   │   ├── hello.html.j2
   │   └─ nginx.conf.j2
   ├── tests
   │   ├── inventory
   │   └─ test.yml
   └── vars
       └─ main.yml
```

Рисунок 5.7. Содержимое роли `nginx`

Можно заметить, что появилась директория `tests`, она предназначена для тестов роли и запуска ее на исполнение и является аналогом директории `molecule`. Также появилась директория `files`, в которой будут храниться статические файлы роли.

Перейдем к созданию роли. Заполним соответствующие файлы данными из секций сложного `playbook` и соответствующие директории ранее созданными файлами.

Это значит, что в директории `tasks` файл `main.yml` должен быть заполнен данными из секции `tasks`. В директории `vars` — из секции `vars`. В директории `handlers` — из секции `handlers`. Также файлы из директорий `files` и `templates` должны быть перемещены в директории `files` и `templates` в папке роли.

Теперь напомним `playbook`, запускающий созданную роль. Для вызова составленной роли в `playbook` используется секция `roles`.

Создадим файл `nginx-role.yml` в рабочей директории `ansible` и заполним его следующим образом:

```
---
- name: Install and config Nginx via Role
  hosts: webserver
  become: yes

  roles:
    - nginx
```

После этого выполним данный `playbook` на серверах:

```
ansible-playbook -i hosts nginx-role.yml
```

## 2.2. Практическая часть

В практической работе необходимо выполнить все шаги из разделов 2.1.1–2.1.7, а также сделать индивидуальное задание.

Для индивидуального задания необходимо сделать следующее:

Написать роль (имя роли — Ваше имя) для запуска сервера `nginx`, написать `playbook` для применения роли, провести тестовый запуск `playbook`, в случае успешного прохождения теста применить `playbook` к серверам.

Необходимо добавить переменную, содержащую ФИО, номер группы и номер варианта. Данная переменная должна выводиться в шаблонный файл `nginx`.

Необходимо установить пакет при помощи модуля `APT`, используемого для установки `nginx` в базовой роли.

Нужно добавить в `playbook` задачу (`task`) по установке пакета согласно варианту:

1. `nano`
2. `imagemagick`
3. `wget`
4. `patch`
5. `php-cli`
6. `mysql-client`
7. `jq`
8. `emacs-nox`
9. `zip`
10. `git`
11. `cowsay`
12. `apache`
13. `postgresql-client`
14. `gpg`
15. `figlet`

В работе должны быть обязательно продемонстрированы следующие скриншоты:

1. Собственная роль.
2. Содержимое файлов по секциям: `tasks`, `handlers` и т.д.
3. Содержимое шаблонного файла `nginx.conf.j2`.
4. Содержимое `playbook.yml` с запуском роли.

Обязательно необходимо проверить, что `nginx` и пакет по варианту установлены. А также переменная с Вашими данными должна выводиться в шаблонный файл `nginx`.

### 2.3. Вывод

Практическая работа позволяет закрепить навыки по автоматизации настройки серверов с помощью `Ansible`: создавать `inventory`-файлы, писать `playbook` для установки программного обеспечения, использовать шаблоны `Jinja2` для конфигураций, работать с переменными и обработчиками.

## **2.4. Контрольные вопросы**

1. Что такое Ansible? Каковы его основные принципы работы?
2. Какие компоненты входят в архитектуру Ansible?
3. Какие задачи можно автоматизировать с помощью Ansible?
4. Что такое Playbook в Ansible? Какой формат у файлов Playbook?
5. Какие основные разделы (sections) присутствуют в Playbook?
6. Что такое inventory-файл и как его создать?
7. Как можно определить хосты, на которых будет выполняться Playbook?
8. Каким образом можно задать задачи в Playbook?
9. Для чего нужны handlers и как их вызывать?
10. Что такое роли (roles) в Ansible? Как они участвуют в организации кода?
11. Какие директории есть в стандартной роли?
12. Как выполнить Playbook на удаленных хостах?
13. Как запустить Playbook в тестовом режиме?
14. Как запустить Playbook только для server-3?
15. Как ограничить выполнение Playbook только для определенной группы хостов?
16. Как можно структурировать файлы Playbook для лучшей организации проекта?
17. Какие методы аутентификации поддерживаются в Ansible для доступа к удаленным хостам?
18. Что такое шаблоны (templates) в Playbook? Как они используются?

## **2.5. Требования к отчету**

По итогу выполнения практической работы необходимо оформить отчет, включающий:

1. Титульный лист.
2. Оглавление.
3. Цели и задачи работы.
4. Выполнение всех шагов из разделов 2.1.1–2.1.7.
5. Выполнение индивидуального задания.
6. Письменные развернутые ответы на все контрольные вопросы.
7. Вывод о проделанной работе.

Отчет о практической работы необходимо загрузить в Систему дистанционного обучения и очно защитить на паре.

## **2.6. Список источников для изучения**

1. Документация Ansible [Электронный ресурс] // Ansible. – Режим доступа: [https://docs.ansible.com/?roistat\\_visit=1719155](https://docs.ansible.com/?roistat_visit=1719155) (дата обращения: 22.04.2025).
2. Статья по работе с Ansible [Электронный ресурс] // Timeweb.cloud. – Режим доступа: <https://timeweb.cloud/blog/ansible-cto-eto-i-kak-ispolzovat> (дата обращения: 22.04.2025).
3. Работа с Ansible-galaxy [Электронный ресурс] // Ansible. – Режим доступа: <https://docs.ansible.com/ansible/latest/cli/ansible-galaxy.html> (дата обращения: 22.04.2025).